

 **QtTinySA.py**
Python

analyse the file and tell me if there is a better way to make it work than all the global variables in the main() function or if having all those globals is "ok" in Python

Worked for 14s >

Yes — **there is a substantially better way**, and in this particular program I would not consider the current amount of global state a good long-term design.

The important distinction is that **using module globals in Python is not inherently wrong**. A small GUI program can quite reasonably have a few application-wide objects. The problem here is that `main()` is effectively acting as a dependency container, and almost every other function reaches into that container implicitly.

For example, `main()` creates roughly **42 objects/variables**, including `QtTSA`, `usbInstr`, `settings`, `bands`, `bandstype`, `numbers`, `phasenoise`, etc. Those then become globals because the functions outside `main()` need to see them. The `Analyser` class alone accesses `QtTSA` about 159 times and `usbInstr` about 60 times.  `QtTinySA`

What I would change

The biggest improvement would be to make `Analyser` **the application/controller object** and put its dependencies on `self`.

You already have the beginnings of exactly that design:

```
class Analyser:
    def __init__(self):
        self.maxF = 12000
        self.memF = BytesIO()
        self.mkr_update_timer = QtCore.QTimer()
```

 Python



 Run

But then `Analyser` immediately reaches outside itself:

```

def setGraphs(self):
    self.phaseNoise = PhaseNoiseGraph(
        phasenoise.ui.plotWidget, ...
    )

    self.timespectrum = SurfaceGraph(
        QtTSA.plot_3D, ...
    )

    self.s0 = SpectrumGraph(
        QtTSA.graphWidget,
        QtTSA.waterfall,
        QtTSA.histogram,
        multiplot,
        100
    )

```

So the class has state of its own, but its dependencies are hidden globals.  QtTinySA

I'd instead make it something like:

```

class Analyser:
    def __init__(self, ui, settings, usb, models, limits, threadpool):
        self.ui = ui
        self.settings = settings
        self.usb = usb
        self.models = models
        self.limits = limits
        self.threadpool = threadpool

        self.maxF = 12000
        self.memF = BytesIO()
        self.mkr_update_timer = QtCore.QTimer()

        self.depth = 50
        self.points = 101
        self.wf_data = np.ndarray(2)

```

Then:

```

def setGraphs(self):
    self.phaseNoise = PhaseNoiseGraph(
        self.settings.phasenoise.ui.plotWidget,
        np.ndarray,

```

```

        np.ndarray,
        1
    )

    self.timespectrum = SurfaceGraph(
        self.ui.plot_3D,
        np.ndarray,
        np.ndarray
    )

    self.s0 = SpectrumGraph(
        self.ui.graphWidget,
        self.ui.waterfall,
        self.ui.histogram,
        self.multiplot,
        100
    )

```

The exact structure could be improved further, but the principle is the important bit.

There are really three different kinds of "globals" in your program

I'd treat them differently.

1. True application infrastructure

Things like:

```
threadpool
```

</> Python



▶ Run

at module level are fairly harmless. `basedir` is effectively a constant, and a shared `QThreadPool` is reasonable application infrastructure. You currently create those at module scope.  QtTinySA

I wouldn't spend much effort eliminating these.

You could make the intention clearer:

```
BASE_DIR = os.path.dirname(__file__)
```

</> Python



▶ Run

although naming them uppercase is optional.

2. Application state

This is where the current design becomes problematic.

For example:

```
QtTSA
usbInstr
settings
presetFreqs
phasenoise
fading
pattern
offset
config
bands
bandstype
bandselect
tracecolours
numbers
...
```

These aren't really "globals" conceptually. They're **components belonging to one running application**.

That's a strong indication they should live inside an application/controller object.

At present, `main()` constructs them and then leaves them in global scope so that functions such as `setPreferences()`, `connectActive()`, `exit_handler()`, etc. can access them.

For example, `Analyser.scan()` accesses `usbInstr`, `settings`, `QtTSA`, and `bandstype` directly.  `QtTinySA`

That makes the dependency invisible:

```
</> Python
```



▶ Run

doesn't tell you that `scan()` actually requires:

```
QtTSA
settings
bandstype
usbInstr
...
```



A constructor makes that relationship explicit.

3. GUI/domain objects that belong together

This is the more interesting part.

You have a large collection of models:

```
bands
bandstype
correction
colours
presetmarker
bandselect
tracecolours
checkboxes
rbwtext
tracetext
markertext
correctiontext
numbers
```



The `main()` function spends hundreds of lines constructing and configuring them.

That strongly suggests another object would be useful, something along the lines of:

```
class AppModels:
    def __init__(self, config):
        self.bands = ModelView(...)
        self.bandstype = ModelView(...)
        self.correction = ModelView(...)
```

</> Python



▶ Run

Then your application becomes conceptually:

```
Application
|
```



```

├── UI
│   ├── main window
│   ├── settings dialog
│   ├── bands dialog
│   └── ...
├── Models
│   ├── bands
│   ├── bandstype
│   ├── correction
│   ├── numbers
│   └── ...
├── USB device manager
└── Analyser
    ├── spectra
    ├── graphs
    ├── markers
    └── measurement state

```

That is a much more natural representation of what your program is actually doing.

The biggest smell isn't actually `main()`

It's the **implicit dependency graph**.

For example, `Limit` itself isn't really independent:

```

class Limit:
    ...
    def create(...):

```

</> Python



▶ Run

So although you instantiate:

```

threshold = Limit(...)
lowF = Limit(    )

```

</> Python



▶ Run

the object doesn't know which graph it belongs to. It secretly assumes there is a global `QtTSA`.  QtTinySA

I'd change that to:

```

class Limit:
    def __init__(self, graph, pen, x, y, movable):
        self.graph = graph
        self.pen = pen
        self.x = x
        self.y = y
        self.movable = movable

    def create(self, dash=False, mark='', posn=0.99):
        self.line = self.graph.addLine(
            self.x,
            self.y,
            movable=self.movable,
            pen=self.pen,
            ...
        )

```

Now:

```

self.threshold = Limit(
    self.ui.graphWidget,
    'cyan',
    None,
    self.settings.ui.peakThreshold.value(),
    movable=True,
)

```

</> Python



▶ Run

That's a small change but a **very significant architectural improvement**.

I wouldn't do a huge rewrite

This is important.

I don't think you need to turn this into a giant dependency-injection framework or introduce dozens of tiny classes.

I'd refactor it incrementally.

Step 1 — Create an Application object

Something like:

```
class Application:
    def __init__(self):
        self.app = QApplication.instance() or QApplication([])

        self.ui = ...
        self.settings = ...
        self.usb = USBdevice()

        self.models = ...
```

</> Python



▶ Run

Then `main()` becomes mostly:

```
def main():
    application = Application()
    application.run()

if __name__ == "__main__":
```

</> Python



▶ Run

That's an enormous improvement by itself.

Step 2 — Give `Analyser` explicit dependencies

Instead of:

```
tinySA = Analyser()

tinySA.setGraphs()
tinySA.setGUI()
```

</> Python



▶ Run

with `Analyser` secretly depending on 20 globals, use:

```
analyser = Analyser(
    ui=ui,
    settings=settings,
    usb=usb,
    models=models,
```

</> Python



▶ Run

```
)
```

Now `Analyser` owns the state associated with analysing/scanning.

Step 3 — Move its current state into `self`

Some of this is already done correctly.

For example:

```
self.depth
self.points
self.wf_data
self.spectra
self.phaseNoise
self.polar
self.timespectrum
self.memF
```

</> Python



▶ Run

are naturally `Analyser` state.

You should continue in that direction.

Step 4 — Move the limits into `Analyser`

Currently:

```
best
maximum
damage
threshold
lowF
highF
```

</> Python



▶ Run

are globals.

I'd make:

```

self.limits = {
    "best": Limit(...),
    "maximum": Limit(...),
    "damage": Limit(...),
    "threshold": Limit(...),
    "lowF": Limit(...),
    "highF": Limit(...),
    "reference": Limit(...),

```

</> Python



▶ Run

or, if you use them frequently:

```

self.best
self.maximum
self.damage
self.threshold
self.lowF
self.highF

```

</> Python



▶ Run

This would clean up a surprising number of methods.

For example, your current `updateMarker()` has:

```

limits = (
    maskFreq,
    highF.line.value(),
    lowF.line.value(),
    threshold.line.value()

```

</> Python



▶ Run

Those three dependencies are currently global.  QtTinySA

With the refactor:

```

limits = (
    maskFreq,
    self.highF.line.value(),
    self.lowF.line.value(),
    self.threshold.line.value(),

```

</> Python



▶ Run

The method now clearly obtains everything from the object it belongs to.

One particularly useful intermediate solution

You don't even have to immediately create lots of constructor arguments.

You could create a single application context:

```
@dataclass
class AppContext:
    ui: object
    settings: object
    usb: USBdevice
    config: QSqlDatabase
    models: ...
```

Python



Run

Then:

```
ctx = AppContext()
```

Python



Run

and:

```
class Analyser:
    def init (self ctx):
```

Python



Run

Then temporarily:

```
self.ctx.usb
self.ctx.settings
```

Python



Run

This is already much better than Python module globals because the dependencies are **explicit and testable**.

However, I wouldn't let `ctx` become a giant "bag of everything" permanently. Once the architecture settles, I'd pull frequently-used dependencies directly onto the relevant classes.

There is also a very nice Qt-specific option

Because this is a PySide6 application, you can make your application controller a `QObject` :

```
class ApplicationController(QtCore.QObject):
    def __init__(self, ...):
        super().__init__()
```

</> Python



▶ Run

Then you can use Qt's normal parent/child ownership and signal/slot mechanisms.

For example:

```
class ApplicationController(QtCore.QObject):
    def __init__(self, ui, usb, settings, parent=None):
        super().__init__(parent)

        self.ui = ui
        self.usb = usb
        self.settings = settings
```

</> Python



▶ Run

That fits the architecture of this program particularly well because you're already heavily using Qt signals, timers and workers.

My assessment of the current code

I'd describe it as:

Area	Current situation
A few module globals	Fine
40+ objects created in <code>main()</code>	Too much

Area	Current situation
Analyser depending on globals	Main architectural problem
GUI widgets as globals	Understandable, but avoidable
Database models as globals	Should be grouped/owned
Limit depending on global GUI	Worth fixing
main() doing application composition	Fine in principle
main() being the implicit dependency container	Not ideal
Incremental refactor worthwhile	Definitely

So I **wouldn't say "globals are bad Python"**. In fact, for a small script, globals can be perfectly pragmatic.

But this program has already grown beyond that point. You have a GUI, hardware abstraction, database models, plotting, worker threads, recording/playback and persistent configuration. At that scale, the globals are starting to obscure the architecture rather than simplify it.

The encouraging thing is that **your** `Analyser` **class is already the natural centre of the application**, so you don't need to invent a completely different architecture. You mostly need to finish the encapsulation that you've already started.

One other thing I noticed: the commented-out old `main()` and the current `main()` are essentially two generations of the same architecture; cleaning that historical residue up would make a refactor much easier to reason about. The current `main()` still constructs all the application components before wiring `tinySA` together.

 QtTinySA

I can next ↪ map every current global to the object it should belong to and sketch a concrete refactored `main()` / Application / Analyser structure without changing the program's behaviour.

Sources



please take the next step. i like the idea of the qt-specific option

Worked for 29s >

I'd do this as a **Qt** `QObject` **application controller**, rather than trying to turn every existing global into a dataclass immediately.

The important distinction is:

- `main()` should **assemble** the application.
- `ApplicationController` should **own the running application state**.
- `Analyser` should own measurement/plotting behaviour.
- dialogs, models, limits and timers should belong to the controller or analyser that actually uses them.
- ordinary helper functions should gradually become controller methods.
- `main()` should end up being almost boring.

1. The target structure

I would aim for this:

```
QApplication
├── ApplicationController : QObject
│   ├── ui
│   ├── dialogs
│   │   ├── settings
│   │   ├── preset_freqs
│   │   ├── file_browse
│   │   ├── phase_noise
│   │   ├── fading
│   │   ├── pattern
│   │   └── offset
│   ├── device
│   │   └── USBdevice
│   ├── analyser
│   │   └── Analyser
│   ├── models
│   │   └── bands
```



```

|   |— bandstype
|   |— correction
|   |— colours
|   |— ...
|
|— limits
|   |— best
|   |— maximum
|   |— damage
|   |— threshold
|   |— low_freq
|   |— high_freq
|   |— reference
|
|— timers
    |— usb_check
    |— auto_run

```

That maps remarkably well onto what your current `main()` is already doing.

The difference is that **the objects have an explicit owner instead of being dumped into the module namespace.**

2. The first class I'd add

Put this immediately before `main()` :

```

class ApplicationController(QtCore.QObject):
    """Owns the lifetime and coordination of the TinySA Qt application."""

    def __init__(self, app):
        super().__init__(app)

        self.app = app

        self._create_ui()
        self._create_dialogs()
        self._create_device()
        self._create_analyser()
        self._create_limits()
        self._configure_gui()
        self._create_models()
        self._connect_application()
        self._create_timers()

    def _create_ui(self):
        loader = CustomLoader()

```

```

        self.ui = loader.load(resource_path("spectrum.ui"), None)

def _create_dialogs(self):
    self.dialogs = QtCore.QObject(self)

    self.dialogs.preset_freqs = CustomDialogue(
        resource_path("bands.ui")
    )
    self.dialogs.settings = CustomDialogue(
        resource_path("settings.ui")
    )
    self.dialogs.file_browse = CustomDialogue(
        resource_path("filebrowse.ui")
    )
    self.dialogs.phase_noise = CustomDialogue(
        resource_path("phasenoise.ui")
    )
    self.dialogs.fading = CustomDialogue(
        resource_path("fading.ui")
    )
    self.dialogs.pattern = CustomDialogue(
        resource_path("pattern.ui")
    )
    self.dialogs.offset = CustomDialogue(
        resource_path("offset.ui")
    )

def _create_device(self):
    self.device = USBdevice()

def _create_analyser(self):
    self.analyser = Analyser(
        ui=self.ui,
        device=self.device,
        dialogs=self.dialogs,
    )

def _create_limits(self):
    self.limits = QtCore.QObject(self)

    self.limits.best = Limit(
        "gold", None, -25, movable=False
    )
    self.limits.maximum = Limit(
        "red", None, 0, movable=False
    )
    self.limits.damage = Limit(
        "red", None, 6, movable=False
    )
    self.limits.threshold = Limit(
        "cyan",
        None,

```

```

        self.dialogs.settings.ui.peakThreshold.value(),
        movable=True,
    )
self.limits.low_freq = Limit(
    "cyan",
    (self.ui.start_freq.value() +
     self.ui.span_freq.value() / 20) * 1e6,
    None,
    movable=True,
)
self.limits.high_freq = Limit(
    "cyan",
    (self.ui.stop_freq.value() -
     self.ui.span_freq.value() / 20) * 1e6,
    None,
    movable=True,
)
self.limits.reference = Limit(
    "yellow", None, -110, movable=True
)

self.limits.best.create(True, ">", 0.99)
self.limits.maximum.create(True, ">", 0.99)
self.limits.damage.create(False, ">", 0.99)
self.limits.threshold.create(True, "<|", 0.99)
self.limits.low_freq.create(True, ">", 0.01)
self.limits.high_freq.create(True, "<|", 0.01)
self.limits.reference.create(True, "<|>", 0.99)

def _configure_gui(self):
    self.ui.graphWidget.setYRange(-112, -20)
    self.ui.graphWidget.setDefaultPadding(padding=0.015)
    self.ui.graphWidget.showGrid(x=True, y=True)
    self.ui.graphWidget.setLabel("bottom", "", units="Hz")

    self.ui.waterfall.setDefaultPadding(padding=0.016)
    self.ui.waterfall.getPlotItem().hideAxis("bottom")
    self.ui.waterfall.setLabel(
        "left", ".", color="#FFF", **{"font-size": "2pt"}
    )
    self.ui.waterfall.invertY(True)

    self.ui.histogram.setDefaultPadding(padding=0)
    self.ui.histogram.plotItem.invertY(True)
    self.ui.histogram.getPlotItem().hideAxis("bottom")
    self.ui.histogram.getPlotItem().hideAxis("left")

    self.dialogs.phase_noise.ui.plotWidget.setYRange(-120, -40)
    self.dialogs.phase_noise.ui.plotWidget.plotItem.showGrid(
        x=True, y=True, alpha=0.5
    )
    self.dialogs.phase_noise.ui.plotWidget.plotItem.setLogMode(x=True)

```

```

        self.dialogs.phase_noise.ui.plotWidget.setLabel(
            "bottom", "Offset Frequency", units="Hz"
        )
        self.dialogs.phase_noise.ui.plotWidget.setLabel(
            "left", "Phase Noise", units="dBc/Hz"
        )

    def _create_models(self):
        ...

    def _connect_application(self):
        ...

    def _create_timers(self):
        self.usb_check = QtCore.QTimer(self)
        self.usb_check.timeout.connect(self.device.probe)
        self.usb_check.start(500)

    def run(self):
        self.ui.show()
        self.ui.setWindowTitle(
            self.app.applicationName() +
            self.app.applicationVersion()
        )

        if self.dialogs.settings.ui.auto_run.isChecked():
            self.auto_run_timer = QtCore.QTimer(self)
            self.auto_run_timer.timeout.connect(self.analyser.run_on_start)
            self.auto_run_timer.start(4000)

        return self.app.exec()

    def shutdown(self):
        self.device.stop()
        self.analyser.shutdown()

```

That is deliberately **not** the final version. It's the architectural boundary I'd establish first.

3. I would make `Analyser` a `QObject`

This is the part of your file where I particularly like the Qt-specific approach.

Currently:

Python



Run

I'd change that to:

```
class Analyser(QtCore.QObject):
    """Controls measurements, plotting and the analyser GUI."""

    def __init__(self, *, ui, device, dialogs, parent=None):
        super().__init__(parent)

        self.ui = ui
        self.device = device
        self.dialogs = dialogs

        self.maxF = 12000
        self.memF = BytesIO()
        self.mkr_update_timer = QtCore.QTimer(self)

        self.dev_ref = []
        self.depth = 50
        self.points = 101
        self.wf_data = np.ndarray(2)
```

Now look at one of your existing methods.

Currently:

```
def setGraphs(self):
    self.phaseNoise = PhaseNoiseGraph(
        phasenoise.ui.plotWidget,
        np.ndarray,
        np.ndarray,
        1
    )

    self.polar = PolarGraph(pattern.ui, 4, 40)

    self.timespectrum = SurfaceGraph(
        QtTSA.plot_3D,
        np.ndarray,
        np.ndarray
    )

    self.s0 = SpectrumGraph(
        QtTSA.graphWidget,
        QtTSA.waterfall,
        QtTSA.histogram,
        multiplot,
```

100

</> Python



Run

It becomes:

```
def setGraphs(self):
    self.phaseNoise = PhaseNoiseGraph(
        self.dialogs.phase_noise.ui.plotWidget,
        np.ndarray,
        np.ndarray,
        1,
    )

    self.polar = PolarGraph(
        self.dialogs.pattern.ui,
        4,
        40,
    )

    self.timespectrum = SurfaceGraph(
        self.ui.plot_3D,
        np.ndarray,
        np.ndarray,
    )

    self.s0 = SpectrumGraph(
        self.ui.graphWidget,
        self.ui.waterfall,
        self.ui.histogram,
        self.multiplot,
        100,
    )
```

And:

```
def setSignals(self):
    self.device.signals.result.connect(self.router)
    self.device.signals.save.connect(self.save_data)
    self.device.signals.error.connect(popUp)
    self.device.signals.progress.connect(self.time_path_indicator)
    self.device.stopped.connect(self.allStopped)
    self.device.update_info.connect(self.set_device_info)
```

</> Python



Run

That is a **massive improvement**.

A reader opening `Analyser` can now see exactly what it depends upon.

4. Move `multiplot` into `Analyser`

This one is particularly clear.

At the moment you have:

```
multiplot = pyqtgraph.GraphicsLayout()
```

</> Python



▶ Run

I'd make it:

```
class Analyser(QtCore.QObject):

    def __init__(self, *, ui, device, dialogs, parent=None):
        super().__init__(parent)

        self.ui = ui
        self.device = device
        self.dialogs = dialogs

        self.multiplot = pyqtgraph.GraphicsLayout()
        self.dialogs.fading.ui.grView.setCentralItem(
            self.multiplot
        )
```

Then:

```
self.s0 = SpectrumGraph(
    self.ui.graphWidget,
    self.ui.waterfall,
    self.ui.histogram,
    self.multiplot,
    100,
```

</> Python



▶ Run

The plotting state belongs to the thing doing the plotting.

5. Limit should stop knowing about the application

This is another architectural smell in the current file.

Currently:

```
class Limit:
    ...
    def create(...):
```

</> Python



▶ Run

So a `Limit` object secretly knows that there is a global called `qtTSA`.

Instead:

```
class Limit:
    def __init__(self, graph, pen, x, y, movable):
        self.graph = graph
        self.pen = pen
        self.x = x
        self.y = y
        self.movable = movable

    def create(self, dash=False, mark="", posn=0.99):
        label = "{value:.1f}" if self.y else ""

        self.line = self.graph.addLine(
            self.x,
            self.y,
            movable=self.movable,
            pen=self.pen,
            label=label,
            labelOpts={
                "position": 0.98,
                "color": self.pen,
                "movable": True,
            },
        )

        self.line.addMarker(mark, posn, 10)

        if dash:
            self.line.setPen(
                self.pen,
                width=0.5,
```

```
style=QtCore.Qt.PenStyle.DashLine,  
)
```

Then:

```
self.limits.best = Limit(  
    self.ui.graphWidget,  
    "gold",  
    None,  
    -25,  
    movable=False,
```

</> Python



▶ Run

Now `Limit` is reusable and testable.

This is exactly the sort of dependency that global variables make unnecessarily opaque.

6. Group the database models

Your `main()` has a second kind of "global explosion":

```
bands  
bandstype  
correction  
colours  
presetmarker  
bandselect  
tracecolours  
checkboxes  
rbwtext  
tracetext  
markertext  
correctiontext  
numbers  
mans
```

I would **not** make fourteen individual controller attributes unless they're genuinely application-wide concepts.

Instead:

```

class AppModels(QtCore.QObject):

    def __init__(self, db, parent=None):
        super().__init__(parent)

        self.config = db

        self.maps = ModelView("mapping", db, ())
        self.bands = ModelView("frequencies", db, ())
        self.bandstype = ModelView("freqtype", db, ())
        self.correction = ModelView("correction", db, (0, 1, 2, 3))
        self.colours = ModelView("SVGColour", db, ())
        self.presetmarker = ModelView("frequencies", db, ())
        self.bandselect = ModelView("frequencies", db, ())
        self.tracecolours = ModelView("trace", db, (0, 1))
        self.checkboxes = ModelView("checkboxes", db, ())
        self.rbwtext = ModelView("combo", db, ())
        self.tracetext = ModelView("combo", db, ())
        self.markertext = ModelView("combo", db, ())
        self.correctiontext = ModelView("combo", db, ())
        self.numbers = ModelView("numbers", db, ())

```

Then code becomes:

```

self.models.bands
self.models.bandstype
self.models.correction

```

</> Python



▶ Run

instead of:

```

bands
bandstype
correction

```

</> Python



▶ Run

This gives you a nice conceptual boundary:

```

models = database/UI models
dialogs = secondary windows
device = hardware
analyser = measurement/plotting behaviour

```

```
limits = graph limits
ui = main window
```

That's much easier to reason about.

7. The `main()` you ultimately want

After the first refactoring pass, I would aim for something roughly this small:

```
def main():
    app = QApplication.instance()

    if app is None:
        app = QApplication([])

    app.setApplicationName("QtTinySA")
    app.setApplicationVersion(" v2.1.x")

    WWBExporter.register()
    WSMExporter.register()

    controller = ApplicationController(app)

    try:
        return controller.run()
    finally:
        controller.shutdown()

if __name__ == "__main__":
    main()
```

That's it.

And importantly, **this isn't merely making** `main()` **shorter for aesthetic reasons.**

The controller now gives the objects a lifetime:

```
ApplicationController
|
├─ device
├─ analyser
├─ dialogs
└─ models
```



```
|— limits
|— timers
```

Qt then has a natural object hierarchy.

8. There is one thing I would *not* do

I wouldn't replace your current globals with this:

```
app_state = {
    "QtTSA": ...,
    "usbInstr": ...,
    "settings": ...,
    "bands": ...,
    "tinySA": ...,
    ...
}
```

</> Python



▶ Run

or:

```
class AppState:
    pass

state = AppState()
state.QtTSA = ...
```

</> Python



▶ Run

That technically removes dozens of globals, but you've effectively created **one giant global variable containing dozens of globals**.

Likewise, I wouldn't make every object a Qt property or `QSettings` entry. There's no benefit.

The useful part of the Qt approach is **ownership and explicit dependencies**, not simply changing the syntax.

9. The migration should be incremental

I would **not** attempt to rewrite all 2,356 lines in one go.

I'd do it in these stages.

Stage 1 — Controller

Introduce:

Python



Run

and move the construction currently in `main()` into it.

At this stage the program should behave identically.

Stage 2 — Analyser

Change:

Python



Run

to:

Python



Run

and inject:

ui
device

Python



Run

Then systematically replace things like:

QtTSA
usbInstr
settings
phasenoise
pattern

Python



Run

with:

```
self.ui
self.device
self.dialogs.settings
self.dialogs.phase_noise
self.dialogs.pattern
```

</> Python



▶ Run

This is where I'd expect the largest reduction in implicit globals.

Stage 3 — Limit

Change:

</> Python



▶ Run

to receive the graph it operates on.

This is a small change with a very nice payoff.

Stage 4 — Models

Move the `ModelView` instances into:

</> Python



▶ Run

This removes another large collection of names.

Stage 5 — callbacks

The current file has functions such as:

```
band_changed()
pointsChanged()
setPreferences()
dialogPrefs()
about()
clickEvent()
connect()
disconnect()
run_on_start()
```

</> Python



▶ Run

Many of these are really **methods of the application controller**, rather than independent functions.

For example:

</> Python



▶ Run

would become:

```
class ApplicationController(QtCore.QObject):
```

```
    def run_on_start(self):
```

</> Python



▶ Run

and your connection becomes:

</> Python



▶ Run

rather than:

</> Python



▶ Run

That is a very Qt-ish pattern.

10. One particularly nice consequence

You currently have things like:

```
def exit_handler():  
    usbInstr...  
    tinySA...  
    config...  
    usbCheck...
```

</> Python



▶ Run

That's a strong signal that this function is really an **application lifecycle method**.

It should become:

```
class ApplicationController(QtCore.QObject):

    def shutdown(self):
        self.device...
        self.analyser...
        self.models...
        self.usb_check.stop()
```

</> Python



▶ Run

Likewise:

```
connect()
disconnect()
settings_clicked()
checkVersion()
```

</> Python



▶ Run

are mostly controller responsibilities.

So the architecture starts to emerge naturally from the code you've already written rather than imposing an artificial MVC/MVVM framework on it.

My recommended end state

I'd therefore aim for approximately:

QtTinySA.py



```
module constants
    BASE_DIR
    THREAD_POOL

Qt helper classes
    CustomTableModel
    CustomLoader
    CustomDialogue
```

```
application classes
    ApplicationController
    AppModels

domain/controller
    Analyser
    Limit

model helper
    ModelView

main()
```

And eventually I'd expect the module-level mutable state to be reduced to approximately:

</> Python



▶ Run

```
THREAD_POOL = QtCore.QThreadPool()
BASE_DIR = os.path.dirname(__file__)
```

Possibly even `THREAD_POOL` can disappear later if it belongs to the particular component using it.

So yes: I think the Qt-specific QObject approach is a very good fit for this particular program. It isn't just a generic "avoid globals" recommendation — your existing use of signals, timers, dialogs, hardware and GUI state makes QObject ownership particularly appropriate.

The next concrete step I'd take is to **actually refactor the current file in Stage 1 + Stage 2**, preserving behaviour: introduce `ApplicationController`, move the current `main()` construction into it, and modify `Analyser` to receive `ui/device/dialogs`. That should remove the biggest and most troublesome globals without forcing a wholesale rewrite.



Is this conversation helpful so far?

